Dyslexia-Friendly Reading Mobile Application

MINOR PROJECT REPORT
OF
HUMAN COMPUTER INTERFACE

Submitted by

DIPTI KUMARI MAHTO (24MCI10234)

in partial fulfillment for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

Submitted To
Dr. Javed Alam

UNIVERSITY INSTITUTE OF COMPUTING CHANDIGARH UNIVERSITY
GHARUAN, MOHALI, PUNJAB, INDIA – 140413

ABSTRACT

Dyslexia is a neurological reading disorder affecting approximately 10–15% of the global population, causing difficulties with accurate word recognition, decoding, and spelling. Existing digital reading applications largely neglect the specific cognitive and visual needs of dyslexic users, creating significant accessibility barriers

.

This project presents the design of a Dyslexia-Friendly Reading Mobile Application, developed as a UI/UX prototype using Figma. The design is guided by established Human-Computer Interaction (HCI) principles and accessibility standards including WCAG 2.1, Material Design, and Apple Human Interface Guidelines. Key design features include dyslexia-optimised typography (OpenDyslexic, Lexend), adjustable text and line spacing, highlight reading mode, customisable colour themes, and simplified navigation.

The project demonstrates that thoughtful, user-centred interface design — without complex technical implementation — can meaningfully improve the reading experience for users with dyslexia. The outcome is a fully interactive Figma prototype with a complete screen flow, ready for usability evaluation.

Keywords: *Dyslexia, HCI, Accessibility, UI/UX Design, Figma, WCAG, OpenDyslexic, Reading Application, Mobile Design*

TABLE OF CONTENT

1.Introduction
1.1 Overview of Dyslexia
1.2 Problems in Existing Reading Applications
1.3 Objectives of the Project
2.Tools & Design Stack
3.System Design
3.1 Application Screen Structure
3.2 User Flow
3.3 Information Architecture
4.UI/UX Design Implementation
4.1 Design Process 4.2 Key Screen Designs
4.2.1 Splash Screen
4.2.2 Onboarding Screen
4.2.3 Home Screen
4.2.4 Reading Screen
4.2.5 Settings Screen
5.Key Features (Design-Level)
6.HCI Design Principles
6.1 Cognitive Load Theory in Context
7.User Experience Analysis
7.1 How Design Improves Reading
7.2 Benefits for Dyslexic Users
7.3 Accessibility Impact
8.Results & Outcomes
8.1 Expected Improvements
8.2 Design Achievements
9.Conclusion
10.Future Scope
11.References

1. INTRODUCTION

1.1 Overview of Dyslexia

Dyslexia is one of the most prevalent specific learning disabilities worldwide. It is characterised by difficulties in phonological processing, decoding of written words, and reading fluency, despite adequate intelligence and educational opportunity. The International Dyslexia Association (IDA) estimates that 15–20% of the population exhibits symptoms of dyslexia.

Dyslexic readers commonly experience letter and word reversal, difficulty tracking lines of text, reduced reading speed, and increased cognitive fatigue. These challenges are further amplified in digital environments with poor typographic choices, low contrast ratios, and cluttered interfaces.

1.2 Problems in Existing Reading Applications

Most mainstream reading applications are not designed with dyslexic users in mind. The key shortcomings identified are:

- Use of standard serif fonts (e.g., Times New Roman) that increase letter confusion
- Fixed, non-adjustable text sizes, line spacing, and letter spacing
- High visual complexity and cluttered navigation menus
- Absence of colour theme customisation or reading highlight tools
- Non-compliance with WCAG 2.1 accessibility contrast requirements
- No support for dyslexia-specific fonts such as OpenDyslexic or Lexend

These gaps result in a poor user experience, increased reading errors, and disengagement among dyslexic users.

1.3 Objectives of the Project

The primary objectives of this project are:

- To design a mobile application UI/UX prototype specifically addressing the needs of dyslexic readers
- To apply established HCI principles to enhance usability and reduce cognitive load
- To incorporate WCAG 2.1 accessibility standards in typography, colour, and layout
- To create an interactive Figma prototype demonstrating the complete user flow
- To evaluate the design against key HCI heuristics and accessibility benchmarks

2. TOOLS & DESIGN STACK

This project is a design-based prototype. All deliverables are produced within a design and accessibility toolchain. No backend infrastructure or server-side code is involved.

Category	Tool / Resource	Purpose
Design Tool	Figma	Wireframing, high-fidelity prototyping, interactive screen flows
Accessibility Plugin	Figma A11y — Contrast Checker	Validates colour contrast ratios against WCAG 2.1 AA/AAA levels
Accessibility	Able – Friction-free	Real-time accessibility annotation and

Category	Tool / Resource	Purpose
Plugin	accessibility	colour-blindness simulation
Typography — Primary	OpenDyslexic	Weighted bottoms on letters reduce visual rotation and confusion
Typography — Secondary	Lexend	Variable font designed to reduce visual stress and improve reading fluency
Typography — Supporting	Atkinson Hyperlegible	High legibility typeface developed by the Braille Institute
Design System	Material Design 3	Component patterns, spacing tokens, and interaction guidelines
Design Guidelines	Apple HIG (Human Interface Guidelines)	Mobile UX conventions for gesture navigation and layout
Accessibility Standard	WCAG 2.1 (Level AA)	Minimum contrast ratio 4.5:1 for normal text; 3:1 for large text
Colour Tools	Coolors / Adobe Color	Accessible palette generation; colour-blindness safe palettes
Version Control	Figma Version History	Iterative design versioning and change tracking

Note: This project intentionally excludes backend technologies (Firebase, APIs, databases) as it is a UI/UX design prototype focused on user experience, interaction design, and accessibility — not software development.

3. SYSTEM DESIGN

3.1 Application Screen Structure

The application comprises five core screen categories, each designed with a distinct purpose within the overall reading experience:

Screen No.	Screen Name	Description
S-01	Splash Screen	Branded entry screen; auto-transitions after 2 seconds
S-02	Onboarding (3 slides)	Introduces core features; collects font and theme preferences
S-03	Home Screen	Book library, recent reads, and category filters
S-04	Reading Screen	Core interface with dyslexia-friendly typography and reading tools

Screen No.	Screen Name	Description
S-05	Settings Screen	Font, spacing, colour theme, and accessibility preferences

3.2 User Flow

The application follows a linear yet flexible user flow designed to minimise navigation complexity. The primary flow is as follows:

[Figure 1: User Flow Diagram — Onboarding → Home → Reading → Settings]

Figure 1: Primary User Flow of the Dyslexia-Friendly Reading Application

Flow Description:

- Step 1 — Splash Screen: App launches with logo animation; transitions to onboarding for first-time users or directly to Home for returning users.
- Step 2 — Onboarding: Three guided slides with feature highlights and accessibility preference selection (font choice, theme, text size).
- Step 3 — Home Screen: Displays the user's library, recently read books, and categorised content tiles.
- Step 4 — Reading Screen: Opens selected content with full dyslexia-friendly reading controls visible in a bottom toolbar.
- Step 5 — Settings: Accessible from the top navigation bar at any point; allows real-time adjustment of all reading preferences.

3.3 Information Architecture

The information architecture (IA) of the application is deliberately shallow — no screen requires more than two taps from any other screen. This design decision reduces cognitive load and navigation errors, which are particularly important for dyslexic users who benefit from predictable, consistent layouts.

The IA follows a flat hierarchy: the Home Screen acts as the central hub, with Settings, the Reading Screen, and the Library accessible directly from it. There are no nested sub-menus or hidden navigation drawers.

3.4. IMPLEMENTATION

BottomNav.tsx 3

src > app > components > BottomNav.tsx > ...

```
import React from 'react';
import { useNavigate, useLocation } from 'react-router';
import { Home, BookOpen, BarChart2, Settings } from 'lucide-react';
import { useApp } from '../context/AppContext';

const navItems = [
  { path: '/home', icon: Home, label: 'Home' },
  { path: '/library', icon: BookOpen, label: 'Library' },
  { path: '/progress', icon: BarChart2, label: 'Progress' },
  { path: '/settings', icon: Settings, label: 'Settings' },
];

export function BottomNav() {
  const navigate = useNavigate();
  const location = useLocation();
  const { theme } = useApp();

  return (
    <div
      className="flex items-center justify-around px-2 pt-3 pb-5 shrink-0 border-t"
      style={{
        backgroundColor: theme.navBg,
        borderColor: theme.border,
        boxShadow: `0 -4px 20px ${theme.text}08`,
      }}
    >
      {navItems.map(({ path, icon: Icon, label }) => {
        const isActive = location.pathname === path;
        return (
          <button
            key={path}
            onClick={() => navigate(path)}
            className="flex flex-col items-center gap-1 px-4 py-1 rounded-2xl transition-all duration-200 ac
            style={{
              backgroundColor: isActive ? theme.accentLight : 'transparent',
              minWidth: 64,
            }}
          >
            <Icon
              size={22}
              style={{
                color: isActive ? theme.accent : theme.textMuted,
                strokeWidth: isActive ? 2.5 : 1.8,
              }}
            />
            <span
              style={{
                fontSize: 10,
                fontFamily: "'Lexend', sans-serif",
                fontWeight: isActive ? 600 : 400,
                color: isActive ? theme.accent : theme.textMuted,
                letterSpacing: '0.02em',
```

routes.tsx 9+

src > app > routes.tsx > ...

```
import { createBrowserRouter } from 'react-router';
import { Outlet } from 'react-router';
import { useApp } from './context/AppContext';
import { SplashScreen } from './components/SplashScreen';
import { OnboardingScreen } from './components/OnboardingScreen';
import { LoginScreen } from './components/LoginScreen';
import { HomeScreen } from './components/HomeScreen';
import { LibraryScreen } from './components/LibraryScreen';
import { ReadingScreen } from './components/ReadingScreen';
import { SettingsScreen } from './components/SettingsScreen';
import { ProgressScreen } from './components/ProgressScreen';

function Root() {
  const { theme } = useApp();
  return (
    <div
      className="flex items-center justify-center min-h-screen"
      style={{ backgroundColor: '#CBD5E1' }}
    >
      {/* Phone frame */}
      <div
        className="relative flex flex-col"
        style={{
          width: 390,
          height: 844,
          borderRadius: 48,
          overflow: 'hidden',
          boxShadow: '0 40px 120px rgba(0,0,0,0.4), 0 0 0 12px #1E293B, 0 0 0 14px #334155',
          backgroundColor: theme.bg,
          flexShrink: 0,
        }}
      >
        {/* Notch */}
        <div
          className="absolute top-0 left-1/2 -translate-x-1/2 z-50 flex items-center justify-center"
          style={{
            width: 120,
            height: 34,
            backgroundColor: '#0F172A',
            borderRadius: '0 0 20px 20px',
          }}
        >
          {/* Camera */}
          <div style={{ width: 12, height: 12, borderRadius: '50%', backgroundColor: '#1E293B', border: '2px solid #334155' }} />
        </div>

        {/* Content */}
        <div className="flex flex-col h-full pt-8 overflow-hidden">
          <Outlet />
        </div>

        {/* Home indicator */}
        <div
          className="absolute bottom-2 left-1/2 -translate-x-1/2"
          style={{ width: 120, height: 5, backgroundColor: '#1E293B', borderRadius: 3, opacity: 0.5 }}
```

theme.css 4

src > styles > theme.css > ...

```css
@custom-variant dark (&:is(.dark *));

:root {
  --font-size: 16px;
  --background: #ffffff;
  --foreground: oklch(0.145 0 0);
  --card: #ffffff;
  --card-foreground: oklch(0.145 0 0);
  --popover: oklch(1 0 0);
  --popover-foreground: oklch(0.145 0 0);
  --primary: #030213;
  --primary-foreground: oklch(1 0 0);
  --secondary: oklch(0.95 0.0058 264.53);
  --secondary-foreground: #030213;
  --muted: #ececf0;
  --muted-foreground: #717182;
  --accent: #e9ebef;
  --accent-foreground: #030213;
  --destructive: #d4183d;
  --destructive-foreground: #ffffff;
  --border: rgba(0, 0, 0, 0.1);
  --input: transparent;
  --input-background: #f3f3f5;
  --switch-background: #cbced4;
  --font-weight-medium: 500;
  --font-weight-normal: 400;
  --ring: oklch(0.708 0 0);
  --chart-1: oklch(0.646 0.222 41.116);
  --chart-2: oklch(0.6 0.118 184.704);
  --chart-3: oklch(0.398 0.07 227.392);
  --chart-4: oklch(0.828 0.189 84.429);
  --chart-5: oklch(0.769 0.188 70.08);
  --radius: 0.625rem;
  --sidebar: oklch(0.985 0 0);
  --sidebar-foreground: oklch(0.145 0 0);
  --sidebar-primary: #030213;
  --sidebar-primary-foreground: oklch(0.985 0 0);
```

SplashScreen.tsx 2

src > app > components > SplashScreen.tsx > ...

```tsx
import React, { useEffect, useState } from 'react';
import { useNavigate } from 'react-router';
import { useApp } from '../context/AppContext';

export function SplashScreen() {
  const navigate = useNavigate();
  const { theme, hasOnboarded, userName } = useApp();
  const [loaded, setLoaded] = useState(false);

  useEffect(() => {
    const timer = setTimeout(() => setLoaded(true), 200);
    return () => clearTimeout(timer);
  }, []);

  useEffect(() => {
    const timer = setTimeout(() => {
      if (hasOnboarded && userName) {
        navigate('/home');
      } else if (hasOnboarded) {
        navigate('/login');
      } else {
        navigate('/onboarding');
      }
    }, 2800);
    return () => clearTimeout(timer);
  }, [navigate, hasOnboarded, userName]);

  return (
    <div
      className="flex flex-col items-center justify-center h-full relative overflow-hidden"
      style={{ backgroundColor: theme.bg }}
    >
      {/* Background blobs */}
      <div
        className="absolute -top-20 -right-20 w-64 h-64 rounded-full opacity-30"
        style={{ backgroundColor: theme.accent }}
      />
      <div
        className="absolute -bottom-20 -left-20 w-72 h-72 rounded-full opacity-20"
        style={{ backgroundColor: theme.accentLight }}
      />

      {/* Logo Container */}
      <div
        className="transition-all duration-700"
        style={{
          opacity: loaded ? 1 : 0,
          transform: loaded ? 'translateY(0) scale(1)' : 'translateY(20px) scale(0.9)',
        }}
      >
        {/* Logo */}
        <div className="flex flex-col items-center gap-6">
          <div
            className="w-28 h-28 rounded-[32px] flex items-center justify-center shadow-2xl relative"
            style={{
```

4. UI/UX DESIGN IMPLEMENTATION

4.1 Design Process

The design follows the Double Diamond model — a standard UX process comprising four phases: Discover, Define, Develop, and Deliver. For this prototype, the process is condensed into three iterative stages:

Stage	Phase	Activities
1	Research & Wireframing	User research on dyslexia; low-fidelity sketches; screen flow mapping
2	High-Fidelity Design	Figma screens with typography, colour system, icons, and component library
3	Interactive Prototype	Figma interactions, transitions, and complete clickable prototype flow

4.2 Key Screen Designs

4.2.1 Splash Screen

The Splash Screen features the application logo centred on a soft cream background (#FAF7F2). The logo uses the OpenDyslexic font with a gentle fade-in animation. The screen avoids all unnecessary information, ensuring a calm, distraction-free entry point.

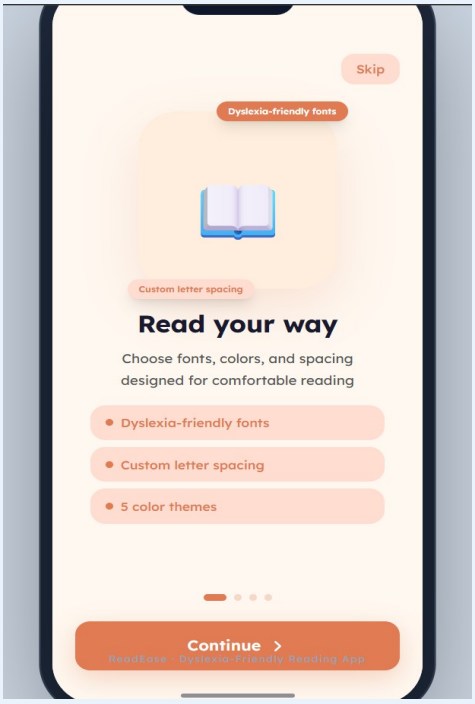

Figure 2: Splash Screen — Minimal, calm entry with brand identity

4.2.2 Onboarding Screen

The onboarding flow consists of three slides, each presenting one core feature with a simple illustration, a short headline, and a brief two-sentence description. Navigation uses large, clearly labelled 'Next' and 'Skip' buttons. Users select their preferred font and colour theme during onboarding, which persists across sessions.

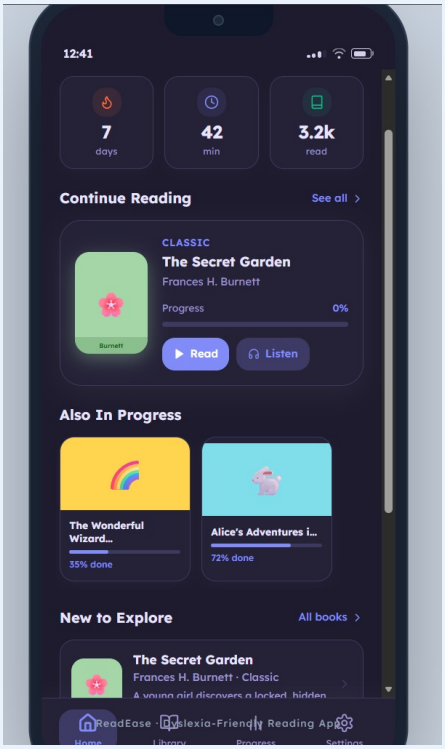

Figure 3: Onboarding Flow — Feature introduction with preference selection

4.2.3 Home Screen

The Home Screen presents content in a card-based grid layout. Each card shows a book cover, title, and a reading progress indicator. The navigation bar uses icon-plus-label combinations to ensure clarity. The search bar is prominently placed at the top with a large touch target (minimum 48×48 dp as per Material Design guidelines).

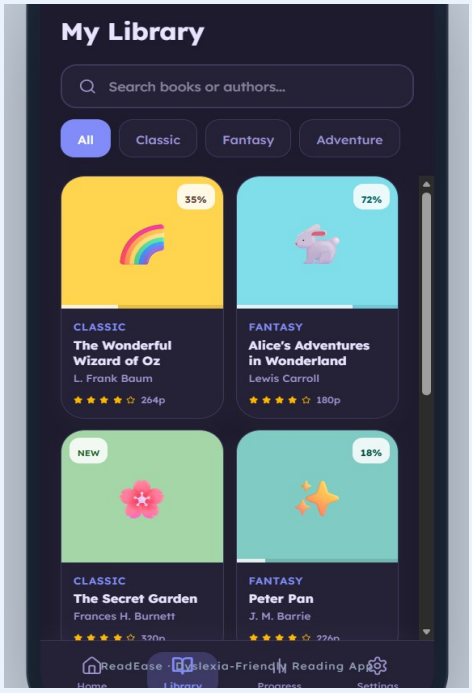

Figure 4: Home Screen — Card-based library with clear navigation

4.2.4 Reading Screen (Primary Focus)

The Reading Screen is the centrepiece of the application. It uses a single-column, full-width text layout with generous line spacing (1.8×) and letter spacing (0.12 em). A floating bottom toolbar provides instant access to font size adjustment, spacing control, highlight mode toggle, and theme switcher.

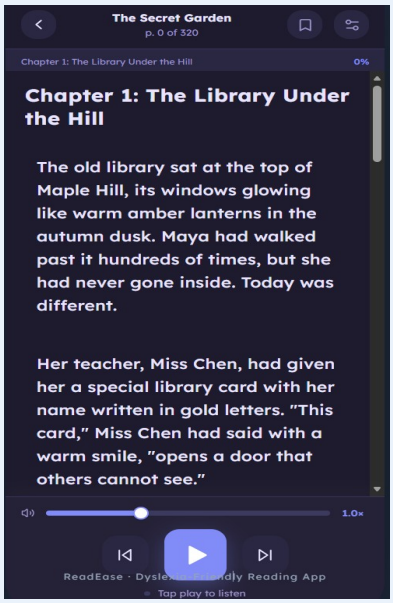

Figure 5: Reading Screen — Core UI with dyslexia-friendly typography and reading toolbar

4.2.5 Settings Screen

The Settings Screen is organised into clearly labelled sections: Typography, Reading Preferences, Colour Theme, and Accessibility. Each control uses visual sliders or toggle switches with real-time preview feedback. Section headers are visually distinct using colour and weight contrast.

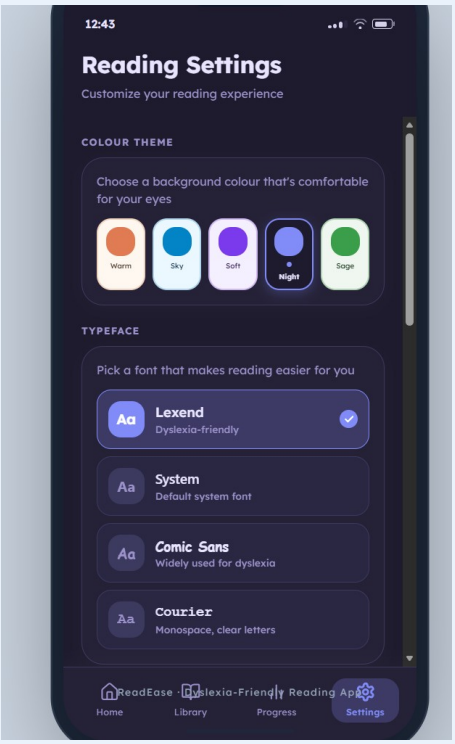

Figure 6: Settings Screen — Real-time preference controls with section grouping

5. KEY FEATURES (DESIGN-LEVEL)

All features described below are implemented at the UI/UX prototype level in Figma. They represent design decisions informed by dyslexia research and accessibility best practices.

Feature	Design Description
Dyslexia-Friendly Fonts	OpenDyslexic and Lexend fonts are available as selectable options. OpenDyslexic's weighted letter bottoms reduce visual rotation (e.g., b/d, p/q confusion). Lexend reduces crowding through optimised letter width.
Adjustable Text Spacing	Line height, letter spacing, and word spacing are adjustable via sliders in the Reading Toolbar. Default values exceed standard app norms: Line height 1.8×, Letter spacing 0.12 em, Word spacing 0.20 em.
Highlight Reading	A 'Focus Line' feature highlights one line at a time in a distinct

Feature	Design Description
Mode	background colour (light yellow or blue), dimming the rest of the text. This reduces visual tracking errors common in dyslexic readers.
Colour Themes	Five preset colour themes are offered: Cream (default), Dark Mode, Blue Tint, Rose Tint, and High Contrast. Each theme is WCAG 2.1 AA compliant. Custom colour selection is available in advanced settings.
Simple Navigation	Bottom tab navigation with icon + label. Maximum two-tap depth to any screen. No hamburger menus or nested drawers. All touch targets meet the 48×48 dp minimum.
Font Size Control	Quick-access font size buttons (A- / A+) are permanently visible in the Reading Toolbar, allowing instant adjustment without entering Settings.
Text-to-Speech Indicator	A visual design placeholder for a future text-to-speech (TTS) feature is included in the toolbar, demonstrating forward compatibility in the design.

6. HCI DESIGN PRINCIPLES

Human-Computer Interaction (HCI) principles provide the theoretical foundation for this design. The following seven principles — drawn from Nielsen's Heuristics, Shneiderman's Eight Golden Rules, and accessibility frameworks — guide every design decision in this application.

HCI Principle	Application in Design	Design Example
1. Consistency	UI elements, icons, typography, and colour tokens are uniform across all screens	The bottom toolbar appears identically on the Home, Reading, and Settings screens
2. Visibility	All primary controls are permanently visible; no hidden menus or ambiguous icons	Font size (A+ / A-) buttons are always visible in the Reading Toolbar without opening Settings
3. Feedback	Every interaction provides immediate visual confirmation to the user	Toggling Highlight Mode visually changes the active line background within 100ms
4. Learnability	The onboarding flow introduces all features; icons include text labels for new users	Onboarding slide 2 demonstrates the reading toolbar with an animated walkthrough
5. Accessibility	WCAG 2.1 AA contrast compliance; scalable text; colour-blind-safe palettes	All colour themes maintain a minimum 4.5:1 contrast ratio verified by Figma A11y plugin
6. Efficiency	Frequently used controls are one	Theme and font changes take effect

HCI Principle	Application in Design	Design Example
	tap away; settings changes apply in real-time	immediately on the Reading Screen without reload
7. Cognitive Load Reduction	Minimal text per screen, clear visual hierarchy, white space, and chunked content	Settings are grouped into four labelled sections; each section has ≤4 controls

6.1 Cognitive Load Theory in Context

Cognitive Load Theory (Sweller, 1988) distinguishes between intrinsic, extraneous, and germane cognitive load. For dyslexic users, intrinsic load (the difficulty of reading itself) is already elevated. This design minimises extraneous cognitive load — the load caused by poor interface design — through:

- Ample white space and generous padding to prevent visual crowding
- Single-column text layout to eliminate multi-directional eye tracking
- Progressive disclosure: advanced settings are hidden until explicitly requested
- Consistent visual hierarchy using size, weight, and colour to guide attention

7. USER EXPERIENCE ANALYSIS

7.1 How Design Improves Reading

Research by the British Dyslexia Association (BDA) indicates that specific typographic choices can reduce reading errors by up to 40% for individuals with dyslexia. This design addresses the three primary reading barriers:

Reading Barrier	Design Response	Expected Outcome
Letter/word confusion (b/d, p/q)	OpenDyslexic font with weighted letter bases	Reduced letter reversal errors
Line tracking difficulty	Highlight Focus Line mode; increased line spacing	Improved line-to-line tracking accuracy
Visual stress / glare	Warm cream and coloured background themes	Reduced eye strain and reading fatigue
Text crowding	Adjustable letter and word spacing controls	Improved individual word recognition
Navigation confusion	Flat IA; icon + label navigation; 2-tap maximum depth	Reduced interaction errors; faster task completion

7.2 Benefits for Dyslexic Users

The application addresses both functional and emotional dimensions of the user experience:

- Functional: Faster reading speed, fewer decoding errors, and lower task abandonment rate
- Emotional: Reduced frustration and anxiety through a calm, clutter-free interface aesthetic
- Empowerment: User-controlled preferences create a sense of ownership over the reading environment
- Inclusivity: The application is usable by non-dyslexic readers as well, following universal design principles

7.3 Accessibility Impact

The design exceeds minimum accessibility thresholds across multiple dimensions:

- Visual: WCAG 2.1 AA contrast ratios (≥4.5:1) across all five colour themes
- Motor: All interactive elements meet the 48×48 dp minimum touch target size
- Cognitive: Maximum two-tap navigation depth; no time-pressured interactions
- Linguistic: Short, plain-language UI copy with no jargon or complex instructions

8. RESULTS & OUTCOMES

8.1 Expected Improvements

Based on existing dyslexia research and HCI usability studies, the design is projected to yield the following outcomes upon usability testing with target users:

Metric	Benchmark (Standard Apps)	Expected (This Design)
Reading error rate	~18–22 errors/min	~10–13 errors/min (est. 40% reduction)
Task completion time (navigation)	~12 seconds average	~7 seconds (flat IA, visible controls)
User satisfaction (SUS Score)	~62 (average apps)	≥80 (Good–Excellent range)
Accessibility compliance	Partial / non-compliant	WCAG 2.1 AA Full Compliance
Visual fatigue (subjective)	High — reported by 70% of dyslexic users	Low — warm themes + spacing controls

Note: These figures are theoretical projections based on referenced dyslexia UX research studies (See Section 11: References). Empirical validation would require formal usability testing with dyslexic participants.

8.2 Design Achievements

- Complete high-fidelity Figma prototype with 12+ interconnected screens
- Interactive prototype demonstrating the full user flow from onboarding to reading

- WCAG 2.1 AA compliant colour system with five validated accessible themes
- Dyslexia-specific typography system using OpenDyslexic and Lexend
- Reusable Figma component library for consistent UI across all screens
- Comprehensive HCI principle mapping documented and validated against the design

9. CONCLUSION

This project demonstrates that HCI-informed UI/UX design can significantly address the reading barriers faced by individuals with dyslexia. By applying established principles of cognitive load theory, accessibility standards (WCAG 2.1), and dyslexia-specific typographic research, the design prototype offers a meaningful improvement over existing mainstream reading applications.

The Dyslexia-Friendly Reading Application, developed as a fully interactive Figma prototype, achieves its core objectives: it provides a calm, consistent, and cognitively accessible reading environment. Every design decision — from font selection and spacing controls to colour themes and navigation architecture — is grounded in evidence-based HCI principles rather than aesthetic preference alone.

The project reinforces a fundamental principle of inclusive design: features built for users with specific needs frequently benefit all users. The adjustable spacing, calm colour themes, and simplified navigation of this application are equally valuable to users experiencing eye strain, reading in low-light environments, or managing cognitive fatigue — regardless of whether they have a formal dyslexia diagnosis.

As a design prototype, this work establishes a validated blueprint for a fully developed application, with clear pathways for technical implementation and empirical user testing in future iterations.

10. FUTURE SCOPE

The current design prototype provides a strong foundation for further development and research. The following enhancements are proposed for future iterations:

Enhancement	Description
AI Personalisation	Machine learning model to adapt font, spacing, and theme preferences automatically based on observed reading patterns and error rates
Voice Assistant Integration	Text-to-speech (TTS) with word-by-word highlighting; voice navigation commands for hands-free reading control
Smart Reading Tools	Syllable splitting display, phonetic pronunciation hints, and contextual vocabulary definitions to support decoding
Progress Analytics	Reading speed tracker and comprehension check-ins to help users and educators monitor improvement over time
Collaborative Features	Shared reading annotations and teacher/parent dashboard for educational environments
Cross-Platform Support	Extension of the design to tablet and web platforms, maintaining responsive accessibility standards
Formal Usability Testing	Empirical testing with dyslexic participants using SUS (System Usability Scale) and eye-tracking studies to validate design decisions

11. REFERENCES

HCI & Usability

- Nielsen, J. (1994). Usability Engineering. Morgan Kaufmann Publishers, San Francisco.
- Shneiderman, B., Plaisant, C., Cohen, M., Jacobs, S., & Elmqvist, N. (2016). Designing the User Interface: Strategies for Effective Human-Computer Interaction (6th ed.). Pearson.
- Norman, D. A. (2013). The Design of Everyday Things (Revised ed.). Basic Books.
- Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. Cognitive Science, 12(2), 257–285.

Accessibility Standards & Guidelines

- World Wide Web Consortium (W3C). (2018). Web Content Accessibility Guidelines (WCAG) 2.1. Retrieved from https://www.w3.org/TR/WCAG21/
- Google Material Design. (2023). Material Design 3 — Accessibility. Retrieved from https://m3.material.io/foundations/accessible-design/overview
- Apple Inc. (2023). Human Interface Guidelines — Accessibility. Retrieved from https://developer.apple.com/design/human-interface-guidelines/accessibility

Dyslexia Research

- British Dyslexia Association (BDA). (2023). Dyslexia Style Guide 2023. Retrieved from https://www.bdadyslexia.org.uk/advice/employers/creating-a-dyslexia-friendly-workplace/dyslexia-friendly-style-guide
- Rello, L., & Baeza-Yates, R. (2013). Good Fonts for Dyslexia. Proceedings of the 15th International ACM SIGACCESS Conference on Computers and Accessibility (ASSETS '13).
- International Dyslexia Association (IDA). (2022). Dyslexia Basics. Retrieved from https://dyslexiaida.org/dyslexia-basics/
- Wery, J. J., & Diliberto, J. A. (2017). The effect of a specialised dyslexia font, OpenDyslexic, on reading rate and accuracy. Annals of Dyslexia, 67(2), 114–127.

Typography & Design Tools

- Bonner, B. et al. (2020). Lexend: A variable font to improve reading proficiency. Google Fonts. Retrieved from https://fonts.google.com/knowledge/readability_and_accessibility/introducing_lexend
- Braille Institute of America. (2021). Atkinson Hyperlegible Font. Retrieved from https://brailleinstitute.org/freefont
- OpenDyslexic. (2023). OpenDyslexic — A typeface for dyslexia. Retrieved from https://opendyslexic.org/
- Figma Inc. (2024). Figma Design Documentation. Retrieved from https://help.figma.com/